

天链人形机器人驱动部分 ROS2 接口说明

(Ver 0.1)

四川天链机器人股份有限公司

[illegible]

目录

1. 机器人控制程序	- 3 -
1.1. 订阅的 ROS2 话题:	- 3 -
1.1.1. 硬件初始化	- 3 -
1.1.2. 启动 controllers	- 4 -
1.1.3. 停止 controllers	- 4 -
1.2. 发布的 ROS2 话题	- 4 -
1.2.1. 机器人状态	- 4 -
2. 机器人 hardware	- 5 -
2.1. 左手 hardware: fdr_huma_robot_leftarm	- 5 -
2.1.1. 控制的硬件:	- 5 -
2.1.2. 关节	- 5 -
2.1.3. 发布的 ROS2 话题	- 6 -
2.1.4. 订阅的 ROS2 话题	- 6 -
2.1.5. 提供的 ROS2 服务	- 7 -
2.2. 右手 hardware: fdr_huma_robot_rightarm	- 8 -
2.2.1. 控制的硬件:	- 8 -
2.2.2. 关节	- 8 -
2.2.3. 发布的 ROS2 话题	- 8 -
2.2.4. 订阅的 ROS2 话题	- 9 -
2.2.5. 提供的 ROS2 服务	- 10 -
2.3. 左脚 hardware: fdr_huma_robot_leftleg	- 11 -
2.3.1. 控制的硬件:	- 11 -
2.3.2. 发布的 ROS2 话题	- 11 -
2.3.3. 订阅的 ROS2 话题	- 11 -
2.3.4. 提供的 ROS2 服务	- 12 -
2.4. 右脚 hardware: fdr_huma_robot_rightleg	- 12 -
2.4.1. 控制的硬件:	- 12 -
2.4.2. 发布的 ROS2 话题	- 12 -
2.4.3. 订阅的 ROS2 话题	- 12 -
2.4.4. 提供的 ROS2 服务	- 13 -
2.5. 腰部 hardware: fdr_huma_robot_torso	- 13 -
2.5.1. 控制的硬件:	- 13 -
2.5.2. 发布的 ROS2 话题	- 13 -
2.5.3. 订阅的 ROS2 话题	- 13 -
2.5.4. 提供的 ROS2 服务	- 13 -
2.6. 头颈部 hardware: fdr_huma_robot_head	- 14 -
2.6.1. 控制的硬件:	- 14 -
2.6.2. 发布的 ROS2 话题	- 14 -
2.6.3. 订阅的 ROS2 话题	- 14 -
2.6.4. 提供的 ROS2 服务	- 14 -
2.7. 脊柱 hardware: fdr_huma_robot_spine	- 14 -
2.7.1. 控制的硬件:	- 15 -
2.7.2. 发布的 ROS2 话题	- 15 -

2.7.3. 订阅的 ROS2 话题	- 15 -
2.7.4. 提供的 ROS2 服务	- 15 -
3. Controllers	- 16 -
3.1. fdr_humarob_rl_controller	- 16 -
3.2. fdr_squat_controller	- 16 -
3.2.1. 发布的 ROS2 话题	- 16 -
3.2.2. 订阅的 ROS2 话题	- 17 -
3.3. fdr_pv_controller	- 17 -
3.3.1. 发布的 ROS2 话题	- 17 -
3.3.2. 订阅的 ROS2 话题	- 18 -

1. 机器人控制程序

机器人运动控制程序模块：`fdr_humarobot`，是启动机器人底层控制器的主程序，实现了机器人的状态管理、实时控制和硬件通信等功能。

程序位置：

`install/fdr_humarobot/lib/fdr_humarobot/fdr_humarobot`

程序参数：

`--init`：初始化 `hardware`，如打开串口，关节通信，关节初始化等

`--start`：启动控制器配置文件中指定的 `controllers`

配置文件：

程序包需要两个主要配置文件：

- 硬件描述文件：``urdf/huma_robot_controller/huma_robot_controller.urdf.xacro``
- 控制器配置文件：``config/huma_robot_control/ros2_controllers.yaml``

机器人 control 线程：

`fdr_humarobot` 启动一个线程用于实现硬件交互和控制器更新。如果启动了 `rt` 内核，该线程调度优先级将会被设置到 50。线程循环周期在“控制器配置文件”中配置。

线程中主要实现了：

Read：读取硬件状态，读取电机的状态，并且通过 `transmission` 关系转换到 `joint`

Update：更新所有 `controller`，调用 `controller` 的 `update` 函数（该函数中实现了一些基础控制算法）。

Write：将 `controller` 中 `update` 计算到的 `joint` 控制命令写入，通过 `transmission` 转化到电机命令。

1.1. 订阅的 ROS2 话题：

1.1.1. 硬件初始化

话题名：`/fdr_humarobot/init_robot`

类型：`std_msgs::msg::String`

功能：初始化机器人硬件，功能同 `fdr_humarobot` 启动时带上 `--init` 参数

`fdr_humarobot` 程序启动时，可以不带任何参数，然后通过本话题进行硬件初始化，然后再使用“启动 `controllers`”话题启动

调用本话题后

1.1.2. 启动 controllers

话题名: /fdr_humarobot/start_robot

类型: std_msgs::msg::String

功能: 加载并启动控制器配置文件中指定的 controllers, 功能同 fdr_humarobot 启动时带上 --start 参数

1.1.3. 停止 controllers

话题名: /fdr_humarobot/stop_robot

类型: std_msgs::msg::String

功能: 停止启动的 controllers

1.2. 发布的 ROS2 话题

1.2.1. 机器人状态

话题名: /fdr_humarobot/status

类型: diagnostic_msgs::msg::DiagnosticArray

功能: 发布机器人状态

发布频率: 1HZ

机器人状态:

status.level: 0: 机器人没有异常, 2: 机器人当前存在异常

status.message: 当前机器人信息, 如: "idle", "initializing", "inited", "starting", "running", "stoping", "error"}

2. 机器人 hardware

机器人硬件层按 `ros2_control` 框架由 `fdr_robot_system` 实现，实现关节、传感器等的硬件驱动功能，提供状态、控制接口。

模块在 `fdr_humarobot` 后初始化时加载(使用了 `--init` 参数, 或者接收到硬件初始化话题)。

`fdr_humarobot` 根据“硬件描述文件”加载对应的 hardware:

- `fdr_huma_robot_leftarm`: 左手
- `fdr_huma_robot_rightarm`: 右手
- `fdr_huma_robot_head`: 头颈部
- `fdr_huma_robot_torso`: 腰部
- `fdr_huma_robot_leftleg`: 左腿
- `fdr_huma_robot_rightleg`: 右腿

2.1. 左手 hardware: `fdr_huma_robot_leftarm`

2.1.1. 控制的硬件:

左手 hardware 模块控制的电机名字及定义如下:

- `motor_arm_left_humeralp`: 前后抬手对应的电机
- `motor_arm_left_humeralr`: 左右抬手对应的电机
- `motor_arm_left_humeraly`: 肩部转动
- `motor_arm_left_elbow`: 手肘
- `motor_arm_left_wristr`: 手腕转动
- `motor_arm_left_wristp`: 手腕并联电机的上面一个
- `motor_arm_left_wristy`: 手腕并联电机的下面一个

2.1.2. 关节

左手关联关节名称为:

- `joint_arm_left_humeralp`
- `joint_arm_left_humeralr`
- `joint_arm_left_humeraly`
- `joint_arm_left_elbow`
- `joint_arm_left_wristr`
- `joint_arm_left_wristp`
- `joint_arm_left_wristy`

2.1.3. 发布的 ROS2 话题

下列话题发布对应电机当前执行的命令

话题名:

/motor_arm_left_humeralp/joint_cmd
/motor_arm_left_humeralr/joint_cmd
/motor_arm_left_humeraly/joint_cmd
/motor_arm_left_elbow/joint_cmd
/motor_arm_left_wristr/joint_cmd
/motor_arm_left_wristp/joint_cmd
/motor_arm_left_wristy/joint_cmd

话题类型:

sensor_msgs::msg::JointState

数据:

position[0]: 位置控制命令, 单位: rad
velocity[0]: 速度控制命令, 单位: rad/s

下列话题发布对应电机当前状态

话题名:

/motor_arm_left_humeralp/joint_states
/motor_arm_left_humeralr/joint_states
/motor_arm_left_humeraly/joint_states
/motor_arm_left_elbow/joint_states
/motor_arm_left_wristr/joint_states
/motor_arm_left_wristp/joint_states
/motor_arm_left_wristy/joint_states

话题类型: sensor_msgs::msg::JointState

数据:

position[0]: 反馈位置, 单位 rad
velocity[0]: 反馈速度, 单位: rad/s
effort[0]: 反馈扭矩
effort[1]: 控制扭矩

话题发布频率由对应硬件 bus.yml 中电机对应 state_pub_period 的参数设定, 单位 us, 默认发布频率 50Hz

2.1.4. 订阅的 ROS2 话题

设置电机 kp:

/motor_arm_left_humeralp/kp
/motor_arm_left_humeralr/kp
/motor_arm_left_humeraly/kp
/motor_arm_left_elbow/kp
/motor_arm_left_wristr/kp

/motor_arm_left_wristp/kp
/motor_arm_left_wristy/kp
话题类型: std_msgs::msg::Float64

设置电机 kd:

/motor_arm_left_humeralp/kd
/motor_arm_left_humeralr/kd
/motor_arm_left_humeraly/kd
/motor_arm_left_elbow/kd
/motor_arm_left_wristr/kd
/motor_arm_left_wristp/kd
/motor_arm_left_wristy/kd
话题类型: std_msgs::msg::Float64

设置电机原点:

话题名称: /arm_left_device_container/set_motor_origin
话题类型: std_msgs::msg::String
数据: data: 要设置原点的电机名称

关节控制:

话题名称: /arm_left_device_container/joint_trajectory
话题类型: trajectory_msgs::msg::JointTrajectory
数据:

- joint_names 要控制的关节列表
- points[0].positions 和 joint_names 对应的关节位置命令
- points[0].velocities 和 joint_names 对应的关节速度命令

注意一旦使用了该话题进行关节控制, 就不能在 controller 中对这些关节进行控制, 否则会造成硬件多控制源问题, 导致控制紊乱。

2.1.5. 提供的 ROS2 服务

以下服务中{motor_name} 为 2.1.1 中电机名字

- 1) 恢复电机, 重新让电机进入使能状态

服务名: /{motor_name}/recover

类型: std_srvs::srv::Trigger

- 2) sdo 读取, 读取电机寄存器参数

服务名: /{motor_name}/sdo_read

类型: canopen_interfaces::srv::CORead

- 3) sdo 写, 写电机寄存器

服务名: /{motor_name}/sdo_write

类型: canopen_interfaces::srv::COWrite

4) 电机停止，电机去使能
服务名: `/motor_name/halt`
类型: `std_srvs::srv::Trigger`

2.2. 右手 hardware: `fdr_huma_robot_rightarm`

2.2.1. 控制的硬件:

右手 hardware 模块控制的电机名字及定义如下:

- `motor_arm_right_humeralp`: 前后抬手对应的电机
- `motor_arm_right_humeralr`: 左右抬手对应的电机
- `motor_arm_right_humeraly`: 肩部转动
- `motor_arm_right_elbow`: 手肘
- `motor_arm_right_wristr`: 手腕转动
- `motor_arm_right_wristp`: 手腕并联电机的上面一个
- `motor_arm_right_wristy`: 手腕并联电机的下面一个

2.2.2. 关节

右手关联关节名称为:

- `joint_arm_right_humeralp`
- `joint_arm_right_humeralr`
- `joint_arm_right_humeraly`
- `joint_arm_right_elbow`
- `joint_arm_right_wristr`
- `joint_arm_right_wristp`
- `joint_arm_right_wristy`

2.2.3. 发布的 ROS2 话题

下列话题发布对应电机当前执行的命令
话题名:

`/motor_arm_right_humeralp/joint_cmd`
`/motor_arm_right_humeralr/joint_cmd`
`/motor_arm_right_humeraly/joint_cmd`
`/motor_arm_right_elbow/joint_cmd`
`/motor_arm_right_wristr/joint_cmd`
`/motor_arm_right_wristp/joint_cmd`
`/motor_arm_right_wristy/joint_cmd`

话题类型:

sensor_msgs::msg::JointState

数据:

position[0]: 位置控制命令, 单位: rad

velocity[0]: 速度控制命令, 单位: rad/s

下列话题发布对应电机当前状态

话题名:

/motor_arm_right_humeralp/joint_states

/motor_arm_right_humeralr/joint_states

/motor_arm_right_humeraly/joint_states

/motor_arm_right_elbow/joint_states

/motor_arm_right_wristr/joint_states

/motor_arm_right_wristp/joint_states

/motor_arm_right_wristy/joint_states

话题类型:

sensor_msgs::msg::JointState

数据:

position[0]: 反馈位置, 单位 rad

velocity[0]: 反馈速度, 单位: rad/s

effort[0]: 反馈扭矩

effort[1]: 控制扭矩

话题发布频率由对应硬件 bus.yml 中电机对应 state_pub_period 的参数设定, 单位 us,
默认发布频率 50Hz

2.2.4. 订阅的 ROS2 话题

设置电机 kp:

/motor_arm_right_humeralp/kp

/motor_arm_right_humeralr/kp

/motor_arm_right_humeraly/kp

/motor_arm_right_elbow/kp

/motor_arm_right_wristr/kp

/motor_arm_right_wristp/kp

/motor_arm_right_wristy/kp

话题类型:

std_msgs::msg::Float64

设置电机 kd:

/motor_arm_right_humeralp/kd

/motor_arm_right_humeralr/kd

/motor_arm_right_humeraly/kd

/motor_arm_right_elbow/kd

/motor_arm_right_wristr/kd

/motor_arm_right_wristp/kd

/motor_arm_right_wristy/kd

话题类型:

std_msgs::msg::Float64

设置电机原点:

话题名称: /arm_right_device_container/set_motor_origin

话题类型: std_msgs::msg::String

数据: data: 要设置原点的电机名称

关节控制:

话题名称: /arm_right_device_container/joint_trajectory

话题类型: trajectory_msgs::msg::JointTrajectory

数据:

- joint_names 要控制的关节列表
- points[0].positions 和 joint_names 对应的关节位置命令
- points[0].velocities 和 joint_names 对应的关节速度命令

注意一旦使用了该话题进行关节控制, 就不能在 controller 中对这些关节进行控制, 否则会造成硬件多控制源问题, 导致控制紊乱。

2.2.5. 提供的 ROS2 服务

以下服务中{motor_name} 为 2.1.1 中电机名字

1) 恢复电机, 重新让电机进入使能状态

服务名: /{motor_name}/recover

类型: std_srvs::srv::Trigger

2) sdo 读取, 读取电机寄存器参数

服务名: /{motor_name}/sdo_read

类型: canopen_interfaces::srv::CORead

3) sdo 写, 写电机寄存器

服务名: /{motor_name}/sdo_write

类型: canopen_interfaces::srv::COWrite

4) 电机停止, 电机去使能

服务名: /{motor_name}/halt

类型: std_srvs::srv::Trigger

2.3. 左脚 hardware: fdr_huma_robot_leftleg

2.3.1. 控制的硬件:

左脚 hardware 模块控制的电机名字及定义如下:

- motor_leg_left_hipy: 髋关节 yaw 电机
- motor_leg_left_hippri: 髋关节 D32 内侧电机
- motor_leg_left_hipro: 髋关节 D32 外侧电机
- motor_leg_left_knee: 膝盖电机
- motor_leg_left_ankleprt: 脚踝上电机
- motor_leg_left_ankleprb: 脚踝下电机

2.3.2. 发布的 ROS2 话题

`/motor_name/joint_states`

`/motor_name/joint_cmd`

`{motor_name}` 为 2.3.1 中定义的电机名, 话题详见 2.1.3

2.3.3. 订阅的 ROS2 话题

`/motor_name/kp`

`/motor_name/kd`

`{motor_name}` 为 2.3.1 中定义的电机名, 话题详见 2.1.4

设置电机原点:

话题名称: `/leg_left_device_container/set_motor_origin`

话题类型: `std_msgs::msg::String`

数据: data: 要设置原点的电机名称

关节控制:

话题名称: `/leg_left_device_container/joint_trajectory`

话题类型: `trajectory_msgs::msg::JointTrajectory`

数据:

- joint_names 要控制的关节列表
- points[0].positions 和 joint_names 对应的关节位置命令
- points[0].velocities 和 joint_names 对应的关节速度命令

注意一旦使用了该话题进行关节控制, 就不能在 controller 中对这些关节进行控制, 否则会造成硬件多控制源问题, 导致控制紊乱。

2.3.4. 提供的 ROS2 服务

```
{motor_name}/sdo_read  
{motor_name}/sdo_write  
{motor_name}/halt  
{motor_name}/recover
```

{motor_name}为 2.3.1 中定义的电机名， 服务详见 2.1.5

2.4. 右脚 hardware: fdr_huma_robot_rightleg

2.4.1. 控制的硬件：

右脚 hardware 模块控制的电机名字及定义如下：

- motor_leg_right_hipy: 髋关节 yaw 电机
- motor_leg_right_hippri:髋关节 D32 内侧电机
- motor_leg_right_hipro:髋关节 D32 外侧电机
- motor_leg_right_knee:膝盖电机
- motor_leg_right_ankleprt:脚踝上电机
- motor_leg_right_ankleprb:脚踝下电机

2.4.2. 发布的 ROS2 话题

```
{motor_name}/joint_states  
{motor_name}/joint_cmd
```

{motor_name}为 2.4.1 中定义的电机名， 话题详见 2.1.3

2.4.3. 订阅的 ROS2 话题

```
{motor_name}/kp  
{motor_name}/kd  
/leg_right_device_container/set_motor_origin  
/leg_right_device_container/joint_trajectory
```

{motor_name}为 2.4.1 中定义的电机名， 话题详见 2.1.4

2.4.4. 提供的 ROS2 服务

```
{motor_name}/sdo_read  
{motor_name}/sdo_write  
{motor_name}/halt  
{motor_name}/recover
```

{motor_name}为 2.4.1 中定义的电机名， 服务详见 2.1.5

2.5. 腰部 hardware: fdr_huma_robot_torso

2.5.1. 控制的硬件:

腰部 hardware 模块控制的电机名字及定义如下:

- motor_torsopr: 腰部 D32 关节左侧电机
- motor_torsoprr: 腰部 D32 关节右侧电机
- motor_torsoy1:腰部旋转电机

2.5.2. 发布的 ROS2 话题

```
{motor_name}/joint_states  
{motor_name}/joint_cmd
```

{motor_name}为 2.5.1 中定义的电机名， 话题详见 2.1.3

2.5.3. 订阅的 ROS2 话题

```
{motor_name}/kp  
{motor_name}/kd  
/torso_device_container/set_motor_origin  
/torso_device_container/joint_trajectory
```

{motor_name}为 2.5.1 中定义的电机名， 话题详见 2.1.4

2.5.4. 提供的 ROS2 服务

```
{motor_name}/sdo_read  
{motor_name}/sdo_write  
{motor_name}/halt  
{motor_name}/recover
```

{motor_name}为 2.5.1 中定义的电机名， 服务详见 2.1.5

2.6. 头颈部 hardware: fdr_huma_robot_head

2.6.1. 控制的硬件:

头颈部 hardware 模块控制的电机名字及定义如下:

- motor_heady: 头左右旋转电机
- motor_headp: 头前后点头电机
- motor_headr: 头左右歪电机

2.6.2. 发布的 ROS2 话题

/ {motor_name} / joint_states

/ {motor_name} / joint_cmd

{motor_name}为 2.6.1 中定义的电机名， 话题详见 2.1.3

2.6.3. 订阅的 ROS2 话题

/ {motor_name} / kp

/ {motor_name} / kd

/ head_device_container / set_motor_origin

/ head_device_container / joint_trajectory

{motor_name}为 2.6.1 中定义的电机名， 话题详见 2.1.4

2.6.4. 提供的 ROS2 服务

/ {motor_name} / sdo_read

/ {motor_name} / sdo_write

/ {motor_name} / halt

/ {motor_name} / recover

{motor_name}为 2.6.1 中定义的电机名， 服务详见 2.1.5

2.7. 脊柱 hardware: fdr_huma_robot_spine

当把头部和腰部的关节全部接在一个总线上时，使用该硬件配置

2.7.1. 控制的硬件：

脊柱包含头部和腰部 hardware 模块控制的电机名字及定义如下：

- motor_heady: 头左右旋转电机
- motor_headp: 头前后点头电机
- motor_headr: 头左右歪电机
- motor_torsoprl: 腰部 D32 关节左侧电机
- motor_torsoprr: 腰部 D32 关节右侧电机
- motor_torsoy1:腰部旋转电机
- motor_torsoy2:腰部旋转电机 2

2.7.2. 发布的 ROS2 话题

```
{motor_name}/joint_states  
{motor_name}/joint_cmd  
{motor_name}为 2.7.1 中定义的电机名， 话题详见 2.1.3
```

2.7.3. 订阅的 ROS2 话题

```
{motor_name}/kp  
{motor_name}/kd  
{head_device_container}/set_motor_origin  
{head_device_container}/joint_trajectory  
  
{motor_name}为 2.7.1 中定义的电机名， 话题详见 2.1.4
```

2.7.4. 提供的 ROS2 服务

```
{motor_name}/sdo_read  
{motor_name}/sdo_write  
{motor_name}/halt  
{motor_name}/recover  
  
{motor_name}为 2.7.1 中定义的电机名， 服务详见 2.1.5
```

3. Controllers

按 `ros2_control` 框架，使用 `controller` 对系统进行控制，可以在 `controller` 中实现关节、机器人状态发布，轨迹规划、推理等控制。

控制器加载时会去使能该控制器用到的关节，控制器停止时会去使能该控制器用到的关节。

除了标准的 `ROS2controller` 可以使用之外，我们提供了以下 `controller`:

`fdr_humarob_rl_controller`: 强化学习模型推理控制器，通过加载强化学习模型进行推理然后将结果发送给底层

`fdr_squat_controller`: 等同于 ROS 提供的 `joint_trajectory_controller`

`fdr_pv_controller`: 转发位置速度信息的控制器（如设备层级控制器可用后可以放弃该控制器）

其他常用控制器:

`joint_state_broadcaster`: 关节状态广播

3.1. `fdr_humarob_rl_controller`

强化学习模型推理控制器

3.2. `fdr_squat_controller`

关节轨迹运动控制器，用于接收上位机发送的离线轨迹，通常配合 `moveit` 使用。

控制器定义在配置文件“控制器配置文件”中见 1。

在其中定义了

- 控制器名称
- 控制器类型(`type`)
- 控制器参数——关节 (`joints`)

当前预定义的，用到该类型 `controller` 名称:

- `arm_left_joint_trajectory_controller`
- `arm_right_joint_trajectory_controller`
- `head_joint_trajectory_controller`
- `torso_joint_trajectory_controller`

需要 `start` 的 `controller`，在 `auto_start_controllers` 中配置

3.2.1. 发布的 ROS2 话题

控制器状态话题:

话题名称: `/{{controller_name}}/controller_state`

话题类型: `control_msgs::msg::JointTrajectoryControllerState`

话题中包含了该控制器使用的关节的参考及真实位置、速度、加速度、`effort`。

话题如: `/arm_left_joint_trajectory_controller/controller_state`

`/arm_right_joint_trajectory_controller/controller_state`

3.2.2. 订阅的 ROS2 话题

控制器状态话题:

话题名称: `/{{controller_name}}/joint_trajectory`

话题类型: `trajectory_msgs::msg::JointTrajectory`

数据:

- `joint_names` 要控制的关节列表
- `points[i].positions`: 轨迹第 `i` 个点中, `joint_names` 对应的关节位置命令
- `points[i].velocities`: 轨迹第 `i` 个点中, `joint_names` 对应的关节速度命令
- `points[i].accelerations`: 轨迹第 `i` 个点中, `joint_names` 对应的关节加速度命令
- `points[i].time_from_start`: 轨迹第 `i` 个点距轨迹起点的时间

话题如: `/arm_left_joint_trajectory_controller/joint_trajectory`

`/arm_right_joint_trajectory_controller/joint_trajectory`

`time_from_start` 参数说明:

- 当应用场景中, `/{{controller_name}}/joint_trajectory` 接收的是离线低频轨迹时, 该参数必须设置为正值, 且该值大小与关节速度成反比, 在设置时需考虑对应关节速度上限。
- 当应用场景中, `/{{controller_name}}/joint_trajectory` 接收的是在线高频轨迹时, 该参数必须设置为 0, 同时该话题中的 `header.stamp.sec` 与 `header.stamp.nanosec` 也必须设置为 0, 此时控制接收到轨迹后将不做插值直接转发到关节电机执行, 因此要求接收轨迹尽可能平滑(点间距/控制周期<关节最大速度), 频率尽可能等于控制周期。

3.3. fdr_pv_controller

速位控制器, 接收上位机针对特定关节的位置和速度控制信息, 在下一个 `ros_control` 同步周期, `controller.update` 后, 发送给对应的关节电机。

如果当前版本的 `fdr_robot_system`, 提供了 `/{{device_container_name}}/joint_trajectory` 接口的话, 建议使用 `fdr_robot_system` 提供的接口。

3.3.1. 发布的 ROS2 话题

控制器状态话题:

话题名称: `/{{controller_name}}/controller_state`

话题类型: `control_msgs::msg::JointTrajectoryControllerState`

话题中包含了该控制器使用的关节的参考及真实位置、速度、加速度、`effort`。

话题如: `/arm_left_joint_posvel_controller/controller_state`

`/arm_right_joint_posvel_controller/controller_state`

3.3.2. 订阅的 ROS2 话题

控制器状态话题:

话题名称: `/ {controller_name} / joint_trajectory`

话题类型: `trajectory_msgs::msg::JointTrajectory`

数据:

- `joint_names` 要控制的关节列表
- `points[0].positions`: `joint_names` 对应的关节位置命令
- `points[0].velocities`: `joint_names` 对应的关节速度命令
- `points[0].accelerations`: `joint_names` 对应的关节加速度命令

话题如: `/arm_left_joint_posvel_controller/joint_trajectory`

`/arm_right_joint_posvel_controller/joint_trajectory`